

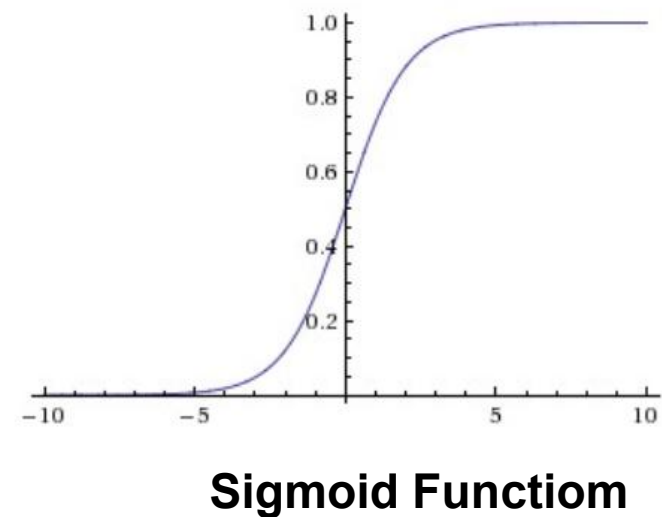
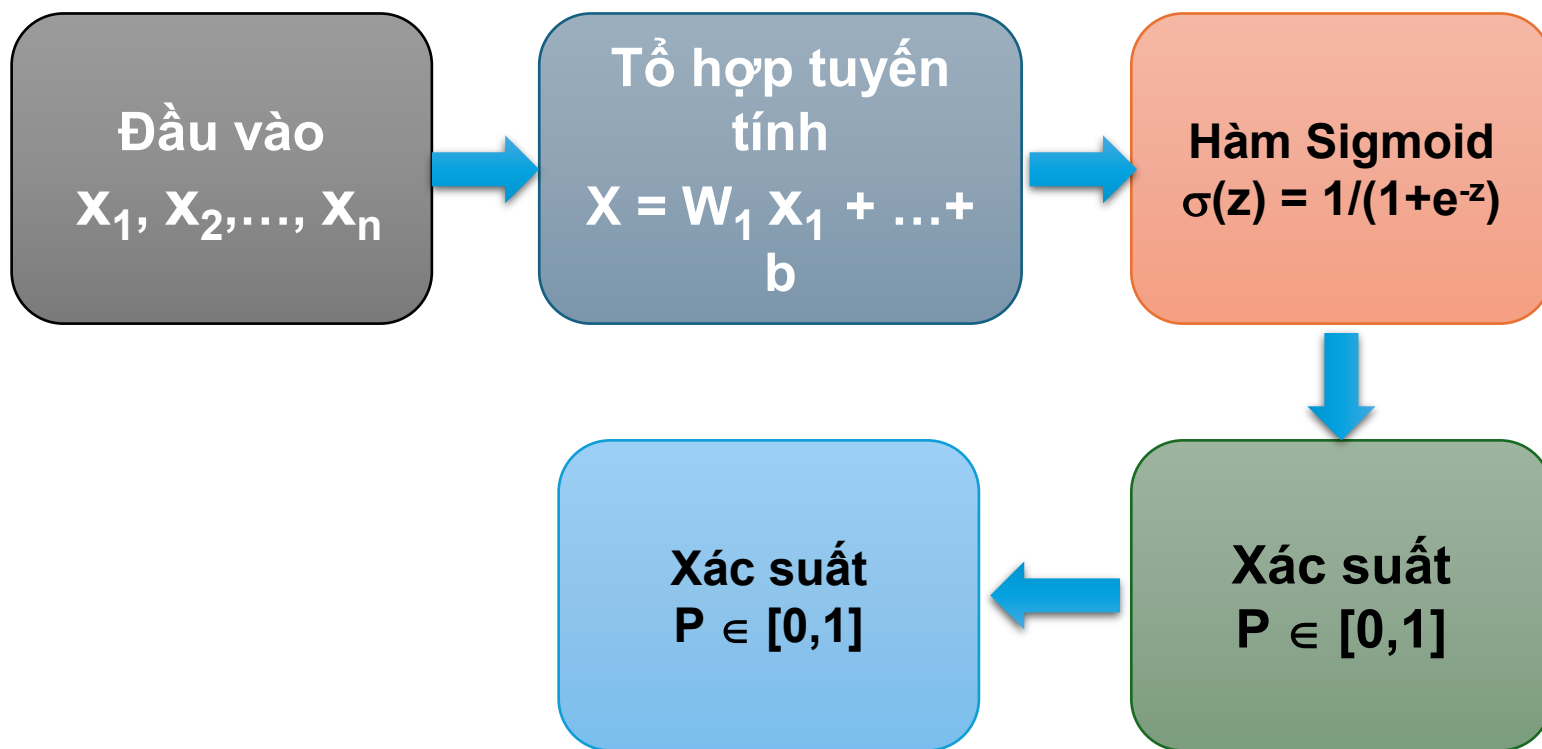
CHƯƠNG 5:
CÁC CHỦ ĐỀ NÂNG CAO ĐỐI VỚI PHÂN TÍCH
DỮ LIỆU LỚN

1. Phân loại và hồi quy dữ liệu lớn

Logistic Regression (Hồi quy Logistic) là một thuật toán học có giám sát (supervised learning) dùng cho bài toán **phân loại** (classification).

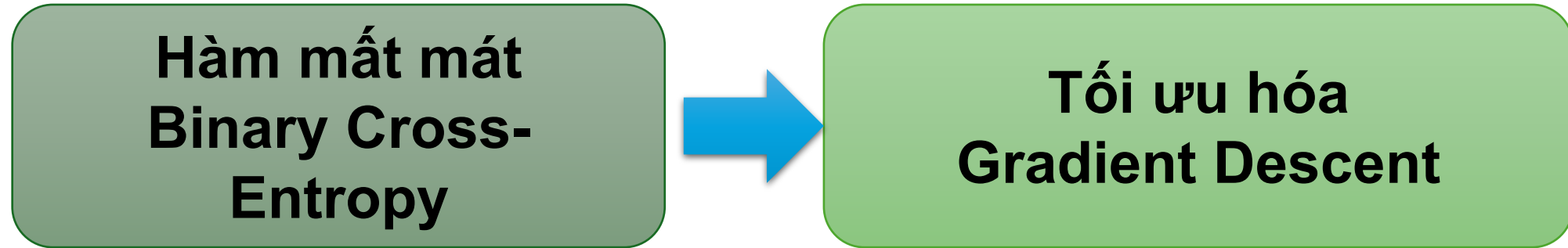
Dự đoán **xác suất** một điểm dữ liệu thuộc về một lớp

Ý tưởng: thuật toán dùng hàm **sigmoid** để đầu ra về khoảng $[0, 1]$, rồi chọn ngưỡng (thường là 0.5) để phân loại.



1. Phân loại và hồi quy dữ liệu lớn

Kỹ thuật & Thuật toán cốt lõi



Tối ưu hóa trọng số:

- **Gradient Descent** (hoặc các biến thể SGD, Adam, L-BFGS): cập nhật w và b theo hướng giảm mất mát
- **Regularization** (L1 — Lasso, L2 — Ridge): giảm overfitting bằng cách phạt trọng số lớn

1. Phân loại và hồi quy dữ liệu lớn

Ưu điểm	Nhược điểm
Đơn giản, dễ hiểu, dễ triển khai	Giả định quan hệ tuyến tính
Huấn luyện nhanh, tốn ít tài nguyên	Nhạy cảm với outlie
Đầu ra là xác suất có ý nghĩa	Tương quan cao dễ dẫn tới sai hệ số
Ít bị overfitting	

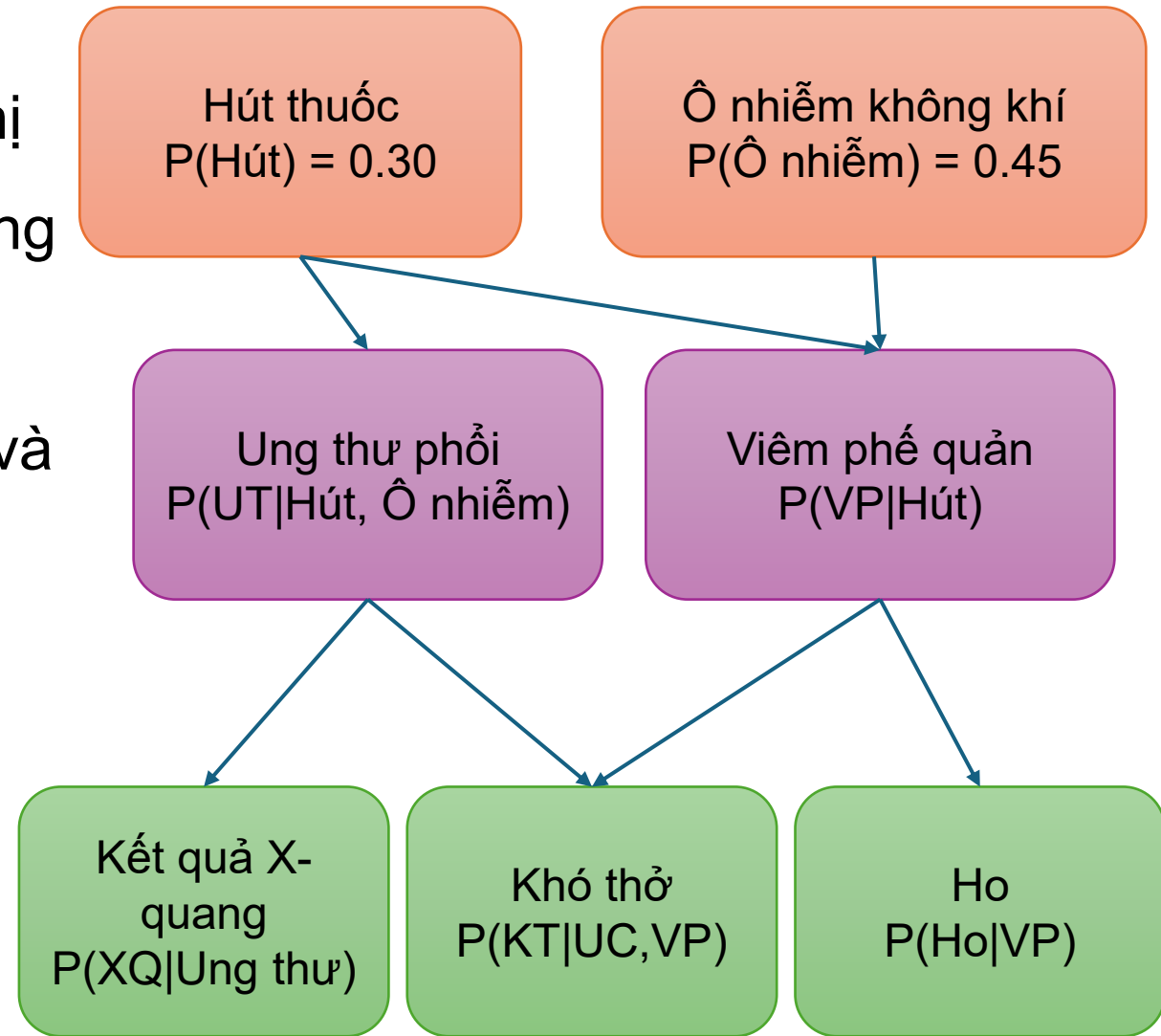
Ứng dụng:

Bài toán có **ranh giới phân tách tuyến tính** (hoặc gần tuyến tính), cần **giải thích được kết quả** (như trong y tế, tài chính)

2. Các mô hình đồ thị xác suất

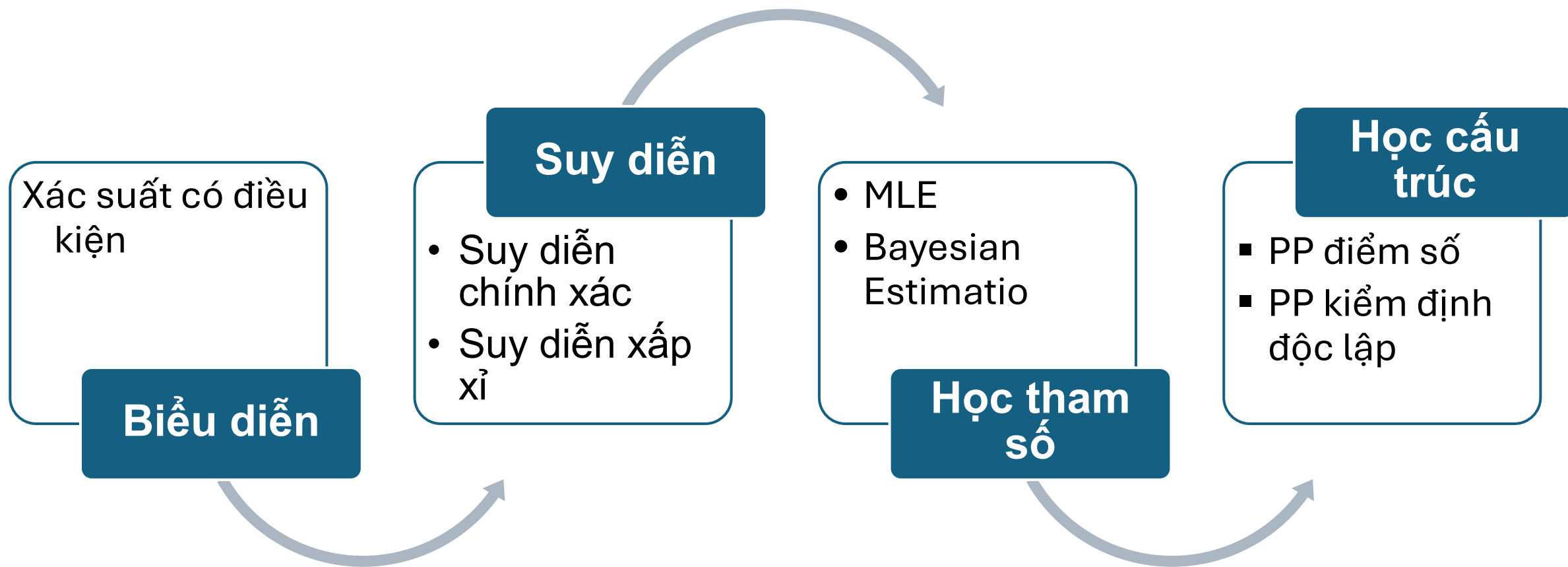
Bayesian Network (Mạng Bayes) — Đồ thị có hướng: mô hình đồ thị xác suất có hướng (Directed Acyclic Graph — DAG), trong đó mỗi **nút** đại diện cho một biến ngẫu nhiên và mỗi **cạnh có hướng** thể hiện quan hệ phụ thuộc nhân quả hoặc điều kiện giữa các biến.

- ❖ Mô hình mã hóa phân phối xác suất kết hợp của tất cả biến một cách hiệu quả thông qua định lý Bayes.



2. Các mô hình đồ thị xác suất

Kỹ thuật & Thuật toán cốt lõi



2. Các mô hình đồ thị xác suất

Ưu điểm	Nhược điểm
Xử lý tự nhiên dữ liệu bị thiếu	Phụ thuộc nhiều vào giả thiết phân phối và cấu trúc được chọn
Có khả năng giải thích được	Khó biểu diễn quan hệ phi tuyến phức tạp
Kết hợp tri thức chuyên gia với dữ liệu	Cần lượng dữ liệu lớn để học cấu trúc đáng tin cậy, đặc biệt với nhiều biến
Suy diễn nhân quả và phân tích	

Ứng dụng: các lĩnh vực đòi hỏi ra quyết định trong điều kiện bất định

- **Y tế & chẩn đoán bệnh:** hệ thống hỗ trợ chẩn đoán (như PATHFINDER cho bệnh máu), dự đoán nguy cơ bệnh lý.
- **Lọc spam và phân loại văn bản:** Naive Bayes (mạng Bayes đơn giản hóa) là nền tảng của nhiều bộ lọc email.
- **Hệ thống nhận dạng giọng nói/hình ảnh:** kết hợp bằng chứng từ nhiều nguồn

3. Giảm gradient ngẫu nhiên

Thuật toán Gradient Descent ngẫu nhiên (Stochastic Gradient Descent – SGD: một biến thể của **Gradient Descent** dùng để tối ưu hàm mất mát (loss function).

Ý tưởng:

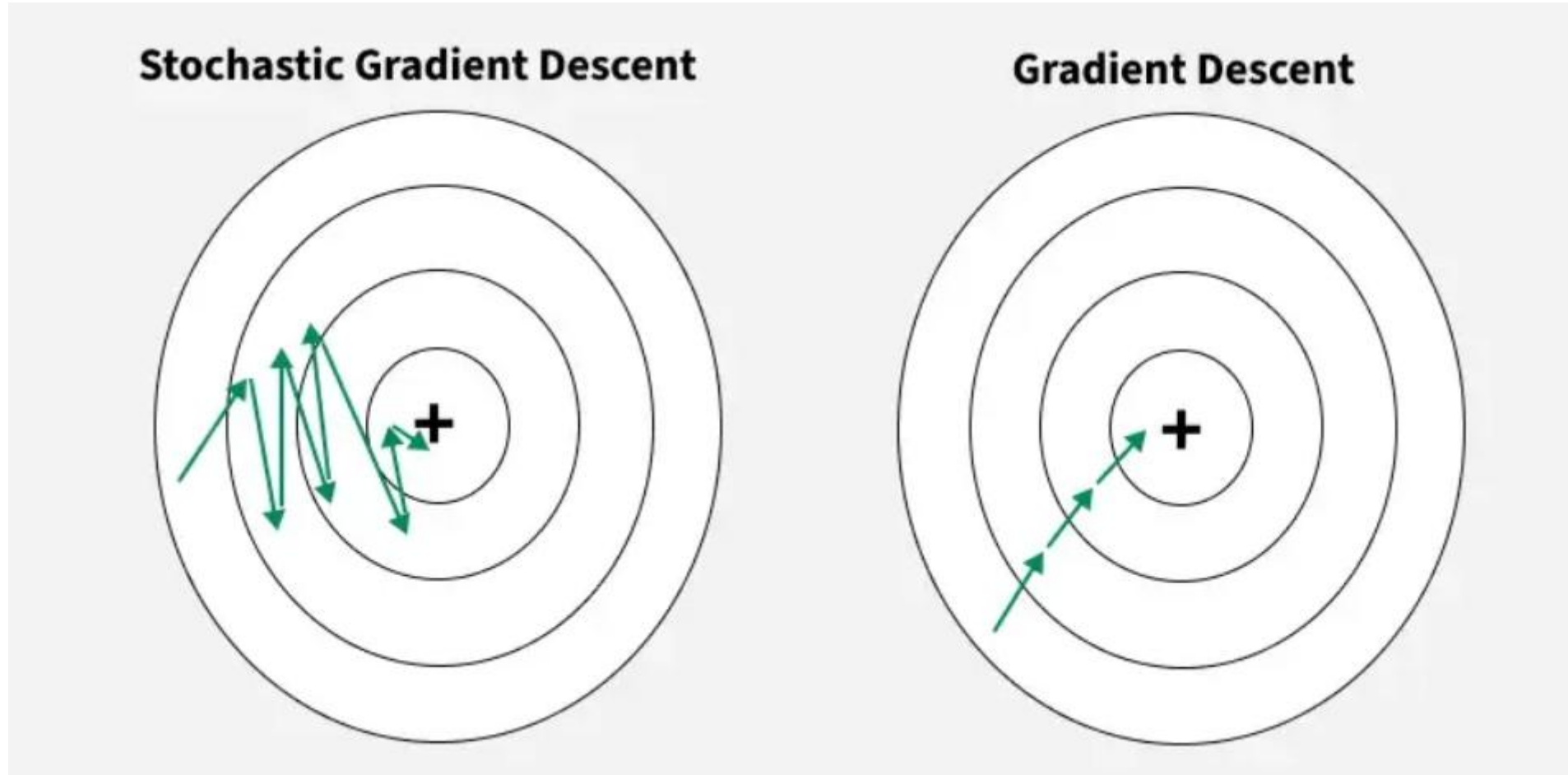
Thay vì tính gradient trên **toàn bộ dữ liệu** (rất tốn thời gian), SGD chỉ dùng **1 mẫu dữ liệu ngẫu nhiên (hoặc một batch nhỏ)** để cập nhật tham số.

$$\theta = \theta - \eta \nabla J(\theta; x_i; y_i)$$

Trong đó:

- θ : tham số mô hình
- η : learning rate
- ∇J : gradient của hàm mất mát theo **một mẫu** (x_i, y_i)

3. Giảm gradient ngẫu nhiên



3. Giảm gradient ngẫu nhiên

Kỹ thuật & Thuật toán cốt lõi

SGD không phải 1 thuật toán đơn lẻ, mà là **một framework tối ưu** (biến thể nâng cấp của SGD):

- Mini-batch SGD: Thay vì 1 điểm $\rightarrow$ dùng một nhóm nhỏ (batch) & Cân bằng giữa tốc độ và độ ổn định
- Momentum: bỏ qua vùng phẳng và giảm dao động
- Nesterov Accelerated Gradient (NAG): Nhìn trước vị trí tương lai trước khi cập nhật
- Adaptive Learning Rate: tự điều chỉnh LR

3. Giảm gradient ngẫu nhiên

Ưu điểm:

- SGD có hiệu suất cao với các tập dữ liệu lớn.
- Thoát khỏi cực tiểu địa phương :
- Học trực tuyến

Nhược điểm:

- Hội tụ nhiều
- SGD rất nhạy cảm với việc lựa chọn tốc độ học
- Thời gian huấn luyện dài

Ứng dụng trong lĩnh vực học máy:

- Huấn luyện mạng nơ-ron sâu
- Xử lý ngôn ngữ tự nhiên (NLP)
- Thị giác máy
- Học tăng cường

4. Mô hình thưa (Sparse Models)

Sparse Logistic Regression (Hồi quy Logistic thưa): một biến thể của hồi quy logistic nhằm tạo ra mô hình có ít tham số khác 0 (sparse). Giúp chọn đặc trưng (feature selection) và tăng khả năng tổng quát hóa.

Hồi quy logistic là mô hình dùng để **phân loại nhị phân**, dựa trên hàm sigmoid:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

4. Mô hình thưa (Sparse Models)

Sparse Logistic Regression = Logistic Regression + **Regularization (L1)**

Ý tưởng:

- Thêm **chuẩn L1 (Lasso)** vào hàm mất mát
- Ép nhiều trọng số $w_i = 0$
- Chỉ giữ lại các đặc trưng quan trọng

Hàm mục tiêu: Hàm loss có dạng:

$$\min_w \sum_{i=1}^n \log \left(1 + e^{-y_i w^T x_i} \right) + \lambda \| w \|_1$$

Trong đó:

- $\| w \|_1 = \sum |w_i|$
- λ : hệ số điều chỉnh độ thưa (sparsity)

4. Mô hình thưa (Sparse Models)

Ưu điểm:

- ✓ Tự động chọn đặc trưng
- ✓ Tránh overfitting
- ✓ Mô hình dễ giải thích (interpretability cao)
- ✓ Phù hợp dữ liệu high-dimensional.

Nhược điểm:

- ✓ Khó tối ưu hơn L2
- ✓ Không ổn định khi feature tương quan cao
- ✓ Phụ thuộc mạnh vào λ
- ✓ Có thể bỏ sót feature hữu ích

Ứng dụng:

- Machine Learning / Data Mining: Feature selection và Giảm chiều dữ liệu
- Sinh học: Chọn gene quan trọng
- NLP (Xử lý ngôn ngữ) Text classification (spam, sentiment) và Dữ liệu cực kỳ sparse (bag-of-words)
- Tài chính: Dự đoán rủi ro (chọn yếu tố quan trọng)

5. Học đồ thị/mạng

Graph Neural Networks (GNN): các phần tử (node) có mối quan hệ với nhau thông qua cạnh (edge), thay vì dữ liệu dạng bảng hay chuỗi thông thường.

GNN mở rộng các mô hình neural network truyền thống:

- Node (đỉnh): đại diện thực thể (người, sản phẩm, website...)
- Edge (cạnh): quan hệ giữa các node
- Graph (đồ thị): tập hợp node + edge

Ý tưởng: Mỗi node học biểu diễn (embedding) bằng cách tổng hợp thông tin từ các node lân cận của nó

5. Học đồ thị/mạng

Kỹ thuật & Thuật toán cốt lõi

GNN hoạt động theo **cơ chế truyền thông tin** (message passing) qua các lớp:

- ✓ B1: Khởi tạo vector đặc trưng cho mỗi node
- ✓ B2: Aggregate (gom thông tin) từ hàng xóm
- ✓ B3: Update (cập nhật) vector của node
- ✓ B4: Lặp lại qua nhiều tầng

Công thức tổng quát:

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \text{AGG} \left(\left\{ h_u^{(k-1)} : u \in N(v) \right\} \right) \right)$$

Trong đó:

- $h_v^{(k)}$:biểu diễn node v tại layer k
- $N(v)$:tập node lân cận
- AGG: hàm tổng hợp (mean, sum, max...)
- σ : hàm kích hoạt

5. Học đồ thị/mạng

Ưu điểm:

- ✓ Khai thác được cấu trúc quan hệ phức tạp
- ✓ Học biểu diễn mạnh mẽ cho graph
- ✓ Áp dụng được cho nhiều domain khác nhau

Nhược điểm:

- Tốn tài nguyên với graph lớn
- Khó mở rộng (scalability)
- Over-smoothing (các node trở nên giống nhau khi nhiều layer)
- Thiết kế kiến trúc phức tạp hơn NN thường

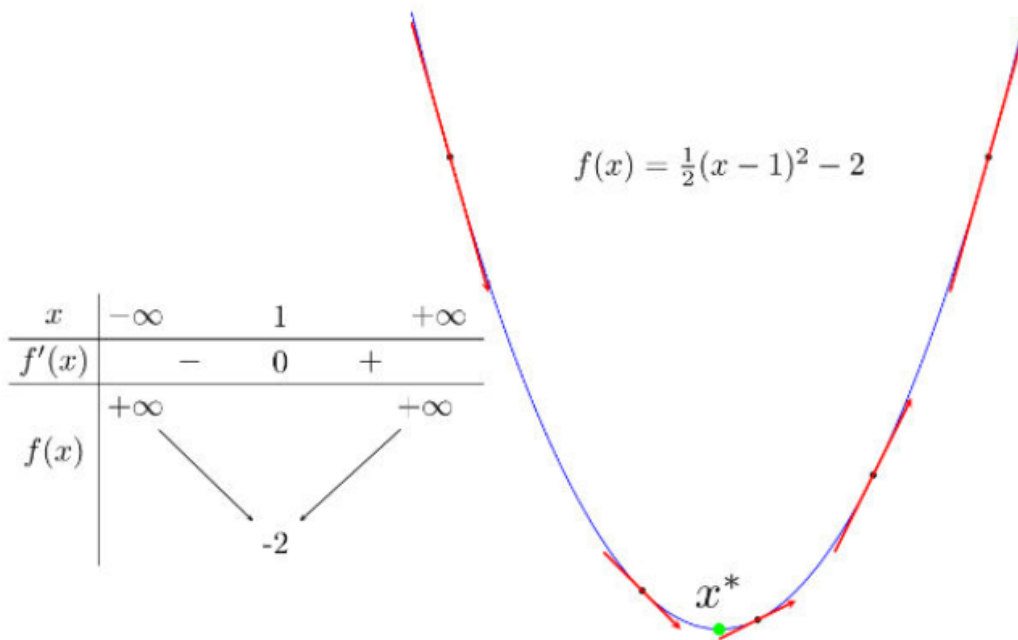
Ứng dụng:

- Data Mining & Big Data: Phân loại node (Node classification), Dự đoán liên kết (Link prediction), Phân cụm đồ thị
- Mạng xã hội: Gợi ý bạn bè, Phát hiện cộng đồng
- Sinh học: Dự đoán cấu trúc protein, Phân tích phân tử
- Thương mại: Gợi ý sản phẩm

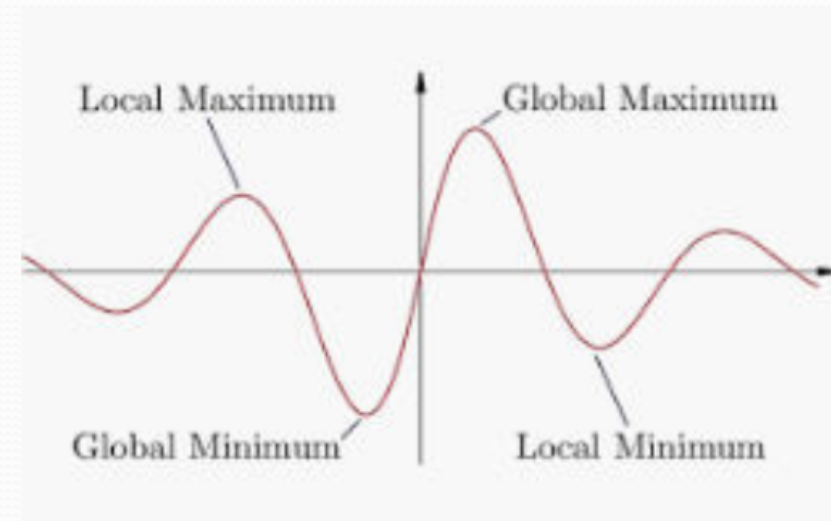
6. Gradient Descent

Gradient Descent: tìm cực tiểu của hàm số.

- Ví dụ:



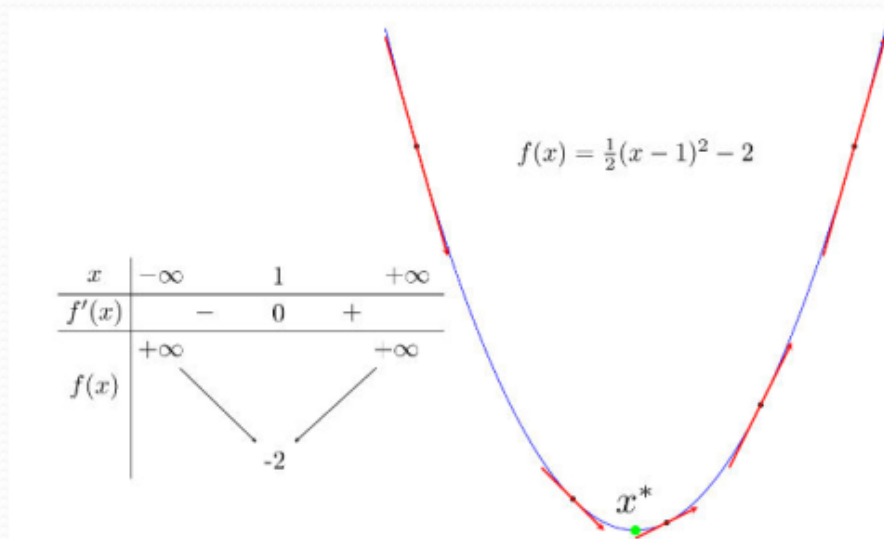
local minimum vs. global minimum



6. Gradient Descent

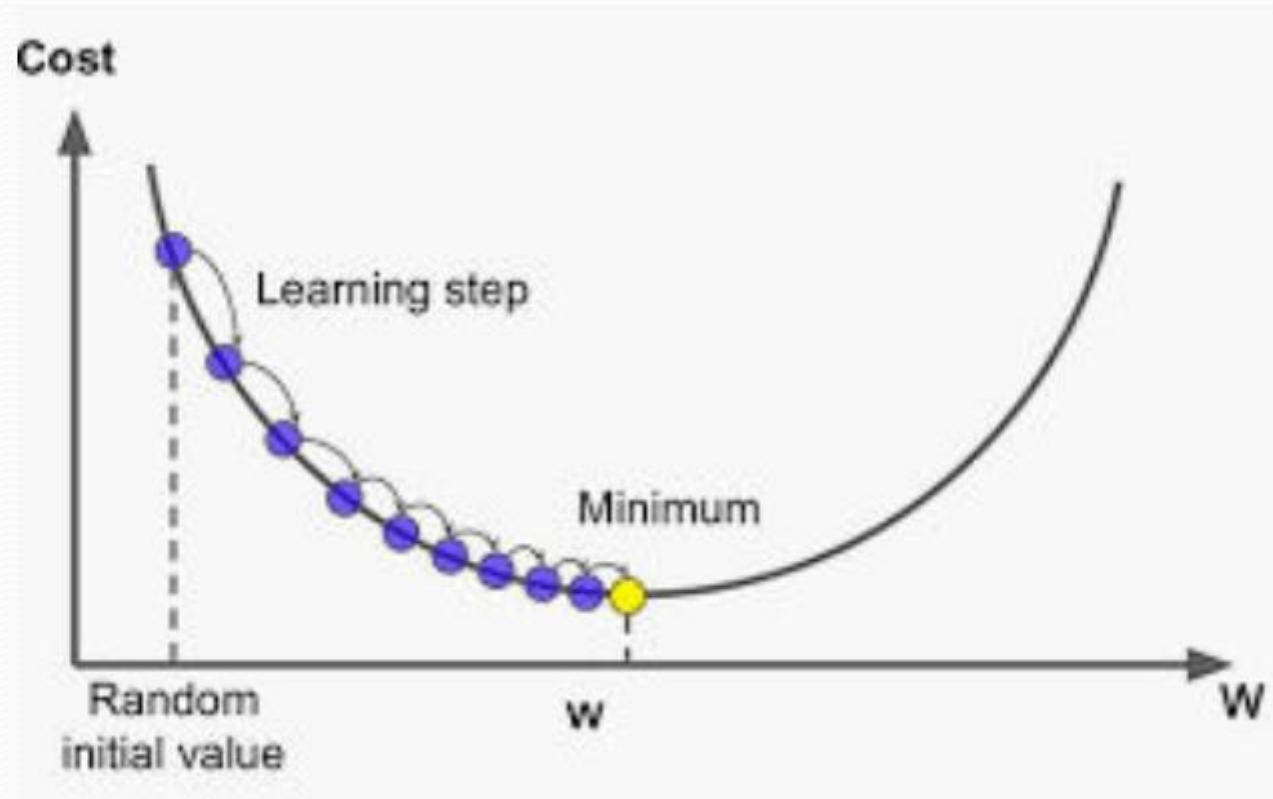
Ví dụ:

- local minimum x^* của hàm số là điểm có đạo hàm $f'(x^*)$ bằng 0
- đạo hàm của các điểm phía bên trái x^* là không dương
- đạo hàm của các điểm phía bên phải x^* là không âm



6. Gradient Descent

- Ví dụ:



1. Phân loại và hồi quy dữ liệu lớn

Gradient Descent cho hàm 1 biến

- Giả sử x_t là điểm ta tìm được sau vòng lặp thứ t . Ta cần tìm một thuật toán để đưa x_t về càng gần x^* càng tốt.
- $f'(x_t) > 0$ thì x_t nằm về bên phải so với x^*
 - Để điểm tiếp theo x_{t+1} gần với x^* hơn, chúng ta cần di chuyển x_t về phía bên trái, tức về phía *âm*
- $f'(x_t) < 0$ thì x_t nằm về bên trái so với x^*
 - Để điểm tiếp theo x_{t+1} gần với x^* hơn, chúng ta cần di chuyển x_t về phía bên phải, tức về phía *dương*.
- **Chúng ta cần di chuyển ngược dấu với đạo hàm:**

$$x_{t+1} = x_t + \Delta$$

Trong đó Δ là một đại lượng ngược dấu với đạo hàm $f'(x_t)$

1. Phân loại và hồi quy dữ liệu lớn

Gradient Descent cho hàm 1 biến

- x_t càng xa x^* về phía bên phải thì $f'(x_t)$ càng lớn hơn 0 (và ngược lại).
- Vậy, lượng di chuyển Δ là tỉ lệ thuận với $-f'(x_t)$
- Do đó:

$$x_{t+1} = x_t - \eta f'(x_t)$$

η là một số dương được gọi là *learning rate*

- Thuật toán gradient descent hoạt động dựa trên nhận xét: Nếu hàm số $f(x)$ xác định và khả vi tại điểm x_t , khi đó giá trị của f sẽ **giảm nhanh nhất** khi đi theo **hướng ngược với gradient** của f
- Nếu ta chọn điểm xuất phát x_0 , và sau đó đi theo công thức trên thì ta sẽ đi dần đến điểm cực tiểu (local minimum) của hàm f (cực tiểu ở đây không chắc là giá trị nhỏ nhất của hàm số).

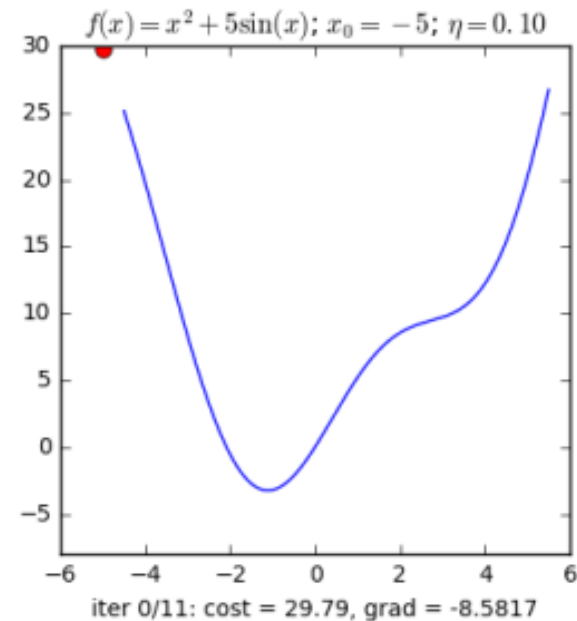
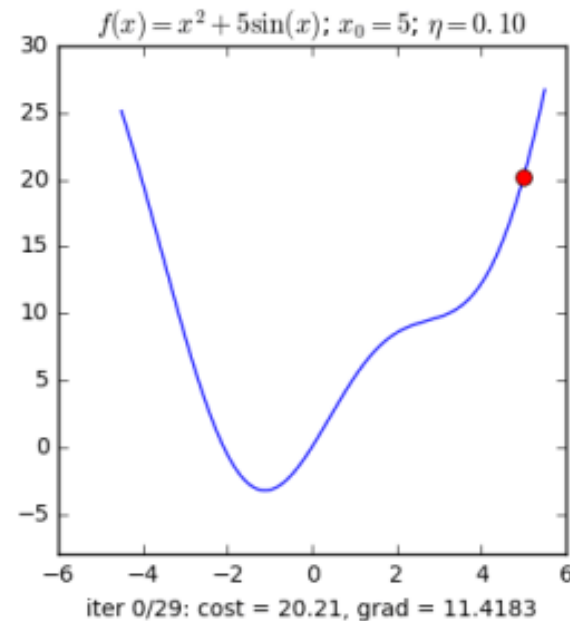
6. Gradient Descent

Gradient Descent cho hàm 1 biến

- Ví dụ: $f(x) = x^2 + 5 \sin(x)$ với đạo hàm $f'(x) = 2x + 5 \cos(x)$
- Giả sử bắt đầu từ một điểm x_0 nào đó, tại vòng lặp thứ t , chúng ta sẽ cập nhật như sau:

$$x_{t+1} = x_t - \eta(2x_t + 5 \cos(x_t))$$

Giá trị khởi tạo khác nhau

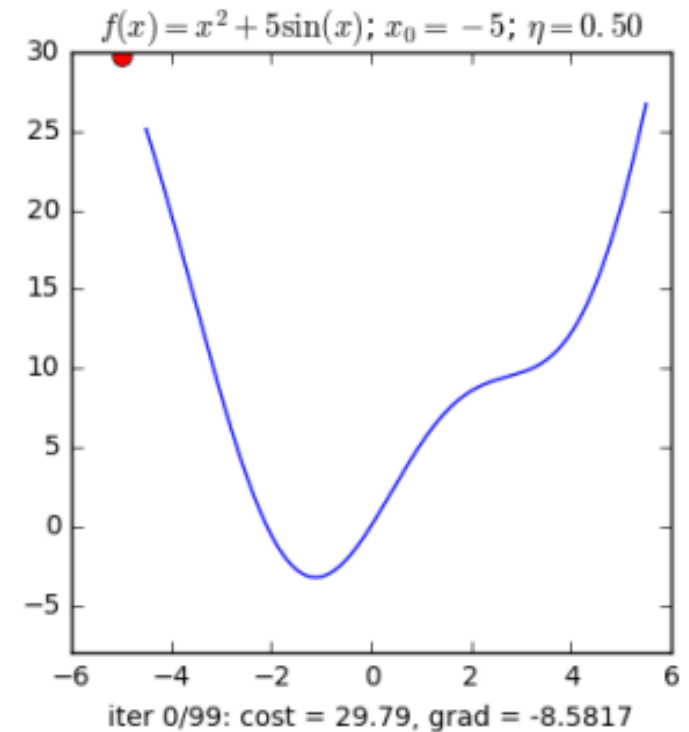
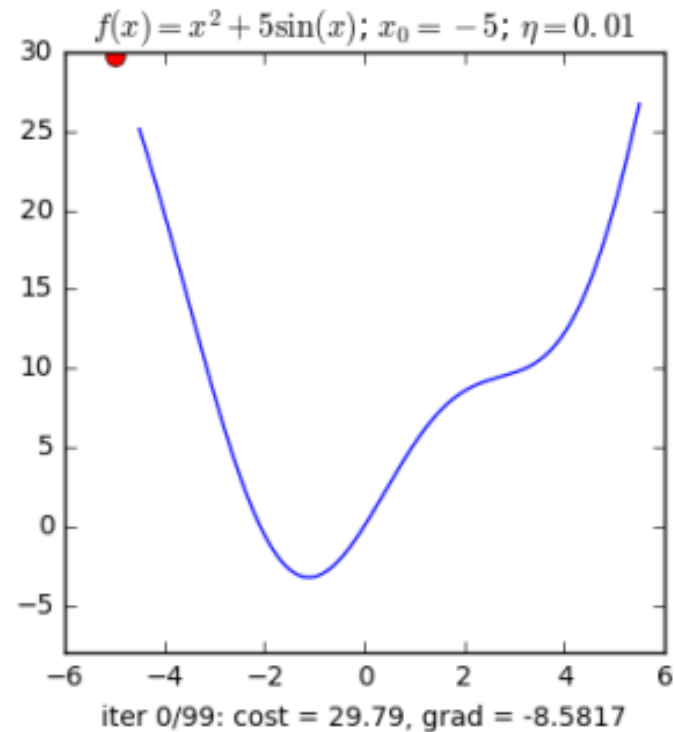


6. Gradient Descent

Gradient Descent cho hàm 1 biến

- Ví dụ:

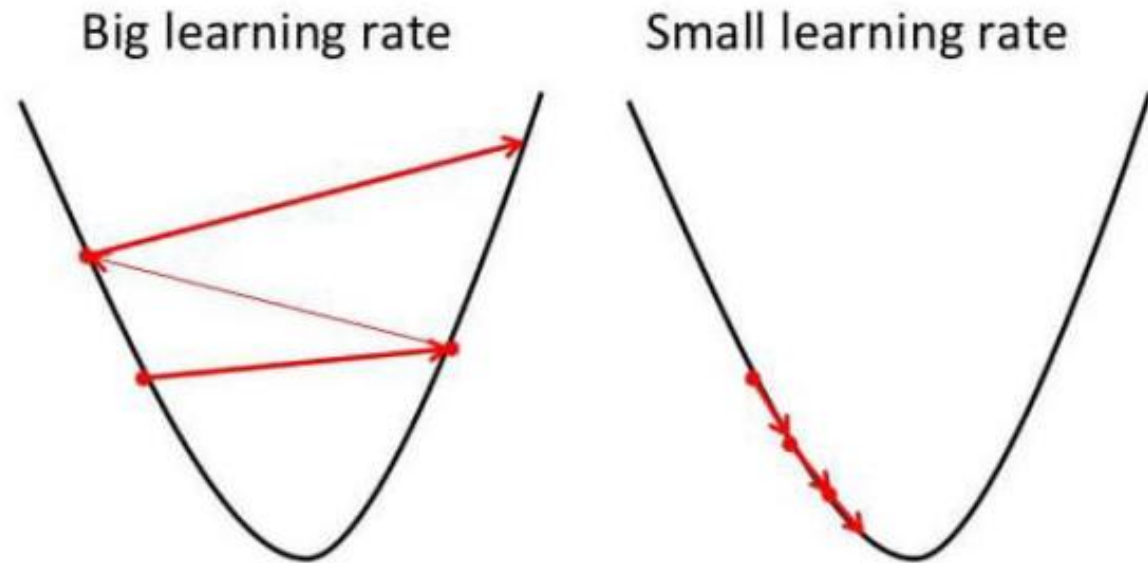
Learning rate khác nhau



6. Gradient Descent

Gradient Descent cho hàm 1 biến

- Learning rate khác nhau



1. Phân loại và hồi quy dữ liệu lớn

Gradient Descent cho hàm 1 biến

- Tổng quát cho hàm nhiều biến: tìm global minimum cho hàm $f(\theta)$ trong đó θ là một vector.
- Thuật toán Gradient Descent

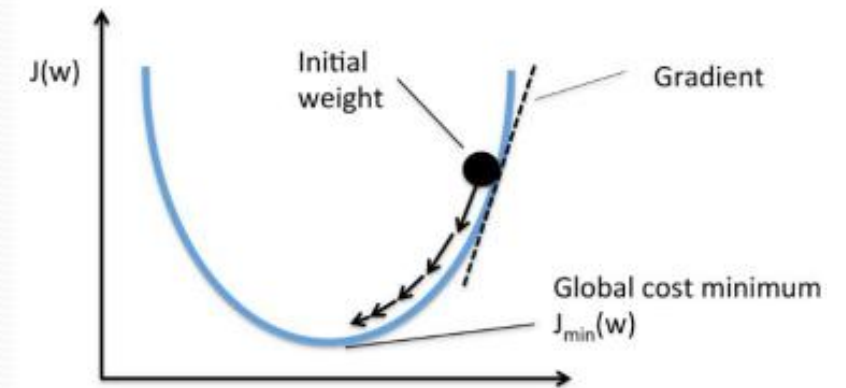
Khởi tạo ngẫu nhiên θ_0 .

Lặp (cho đến khi hội tụ hoặc số lượng vòng lặp vượt quá một ngưỡng)

{

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta_t)$$

}



1. Phân loại và hồi quy dữ liệu lớn

Từ GD cơ bản, người ta đã phát triển nhiều biến thể thông minh hơn:

- **Momentum:** Tích lũy vận tốc từ các bước trước, giúp vượt qua saddle points
- **AdaGrad:** Điều chỉnh learning rate riêng cho từng tham số
- **RMSProp:** Cải tiến AdaGrad, tránh learning rate giảm quá nhanh
- **Adam:** Kết hợp Momentum + RMSProp, hiện là optimizer phổ biến nhất trong Deep Learning

Thank you
